Reusable Assets

A collection of developer tools built with Angular and the H5 SDK for Infor M3 Cloud.

Get Started View on GitHub

Overview

Reusable Assets is an Angular 18 application that provides productivity tools for M3 developers and consultants. It integrates with M3 APIs, ION, Event Hub, and XtendM3 to streamline common development workflows.

Key Features

Feature	Description
XtendM3 CRUD Generator	Generate full CRUD transactions for custom tables with editable code preview modal and upload directly to M3
Event Hub Formatter	Browse, filter, and export Event Hub subscriptions with smart selection sorting
Custom List Checker	Validate custom list configurations from ZAP files or MEC mappings, with bulk config export
MEC Mapping Viewer	Browse MEC mappings with Java source code viewer
Dataflow List	Extract ION dataflow dependencies and generate DES-030 documents
Object Schema Editor	Browse, create, and edit ION noun XSD definitions and metadata (ID XPath, verbs) with GenAI-assisted generation
MEC Variable Validations	Generate null-safe trim statements from MEC Java classes
GenAI Chat	AI chat interface powered by Infor's GenAI service
Kiro AI Chat	AI-powered chat assistant with WSL backend
Custom BOD Generator	Generate CUSBOD mapping templates (.zap) for MEC import with GenAI-assisted noun property generation
Generators	Quick timestamp, UUID, and H5 Script Link generators

Log Configuration	View and manage EC/MEC logger levels directly from the app
Export Configurations	Automate M3 configuration exports (CMS015/CMS010/CRS021/MDBREADMI/CMS047) with bulk export modal and file download
MEC Map Download	Download MEC mappings as .zap files via the IEC MapGen REST API
Session Cache	Cache Grid API results in sessionStorage with tenant-scoped keys for instant loading

---

Contributors

- 

Tech Stack

- **Framework:** Angular 18.2 with TypeScript
- **UI Library:** IDS Enterprise (Soho) via `ids-enterprise-ng`
- **Navigation:** Custom Module Nav sidebar with Angular Router (hash routing)
- **Theming:** IDS Personalize with Light/Dark/Contrast modes and 9 color options
- **M3 Integration:** `@infor-up/m3-odin` and `@infor-up/m3-odin-angular`
- **Code Editor:** Monaco Editor (theme-synced with app mode)
- **Build Tool:** Angular CLI

Getting Started

Set up your development environment and run the application.

Prerequisites

Table of contents

Required Software

Software	Version	Purpose
Node.js	18+	Runtime
Angular CLI	18.2+	Build & serve
npm	9+	Package management

M3 Environment

The application requires access to an Infor M3 Cloud Edition environment with:

- ION API Gateway access
- M3 API (MI) endpoints
- Event Hub subscriptions (for Event Hub Formatter)
- XtendM3 Extensibility API (for CRUD Generator)
- Grid API (for MEC Mapping Viewer)

Optional: Kiro Chat Backend

The Kiro Chat tab requires a local backend running in WSL. See Kiro Chat Setup for details.

Installation

1. Install ODIN CLI

```
npm install -g @infor-up/m3-odin-cli
```

2. Verify Installation

```
odin -h
```

3. Install Angular CLI

```
npm install -g @angular/cli
```

4. Install project dependencies

```
cd reusable-assets
npm install
```

5. Download H5 SDK ION API file

Download the `.ionapi` file from your M3 tenant (ION API → Authorized Apps → Download Credentials).

6. Connect to M3

```
odin login -c <ion-api-file>
```

Running the Application

```
odin serve --multi-tenant --ion-api
```

Or the shorthand:

```
odin serve -mi
```

Navigate to http://localhost:8080.

Local ION API Access (Optional)

For features that call ION API directly (Export Configurations download, Object Schema Editor, GenAI Chat), you need a Bearer token when running locally:

1. Open your M3 H5 in the browser and log in
2. Open DevTools → Network → find any request with an `Authorization: Bearer` header
3. Copy the full token
4. Paste it in `src/environments/environment.ts`:

```
export const environment = {
  production: false,
  ionApiDevToken: 'eyJraWQ...',  // Your token here
};
```

This file is gitignored. Tokens expire after ~2 hours. When deployed to H5, the token comes from the Grid session automatically — no manual config needed.

Building for Deployment

```
odin build
```

The build artifacts will be in the `dist/` directory, ready to upload as an H5 SDK widget.

Running Documentation Locally

Requires Ruby (3.2+) installed and on your PATH.

```
cd docs-site
gem install bundler
bundle install
bundle exec jekyll serve
```

Open http://localhost:4000.

To generate a PDF of the documentation:

```
cd docs-site
npm run pdf
```

Features

Detailed documentation for each tool available in Reusable Assets.

XtendM3 CRUD Generator

Generate full CRUD transactions for XtendM3 custom tables and upload them directly to M3.

Table of contents

Overview

The XtendM3 CRUD Generator creates complete Add, Delete, Get, List, and Update transactions for custom dynamic tables. It handles code generation, direct upload to the M3 Extensibility API, and table definition management.

Capabilities

Code Preview

- **Preview Code** button opens a full-screen modal overlay (90vw × 85vh)
- Tabbed interface showing all 5 generated Groovy transactions (Add, Del, Get, Lst, Upd) plus the table JSON
- Syntax highlighted with Java/Groovy language support

- **Editable** — modify the generated code directly in the editor; changes are used by Upload and Download
- Modified tab indicator (blue dot) shows which transactions have been manually edited
- Upload and Download buttons in the modal header for direct actions from the preview
- Click outside the modal to close it
- Validates configuration before generating preview

Transaction Generation

- Generates **Add**, **Del**, **Get**, **Lst**, and **Upd** transactions
- Configurable field definitions (name, type, length, mandatory, key)
- Auto-generates proper XtendM3 Groovy code with database operations

Upload to M3

- Upload transactions directly to the M3 Extensibility API
- Auto-activation after upload
- Upload table definitions to Foundation REST `importTables` endpoint (with CSRF token handling)

Configuration Management

- Save/load field configurations to M3 environment via `EXT999MI`
- Download generated code as ZIP archive
- Export/import JSON configurations for sharing

User Context

- User field auto-populated from M3 login
- Read-only when authenticated within M3 session

Usage

1. Define your table fields (name, type, length, key fields)
2. Click **Generate** to preview the CRUD transactions
3. Use **Upload** to push directly to M3, or **Download ZIP** for offline use
4. Optionally save your configuration for reuse via **Save Config**

Credits

Originally developed by Nixon Ong.

Event Hub Formatter

Browse, filter, and export Event Hub subscription events from your M3 environment.

Table of contents

Overview

The Event Hub Formatter connects to your M3 environment's Event Hub to browse subscriptions, query events with filters, and export results.

Capabilities

Subscription Browsing

- Browse Event Hub subscriptions directly from the connected M3 environment
- Multi-select subscriptions for batch querying
- Selected subscriptions automatically sort to the top of the dropdown list for easy access

Filtering

- Start time and time span configuration
- Criteria-based filtering on event properties

Results Display

- Filterable and sortable datagrid for event results
- Row selection opens event detail in a Soho modal dialog
- Flattened element columns for easy reading

Export

- Export events to CSV with flattened element columns
- Existing CSV/text formatting preserved for compatibility

Usage

1. Select one or more subscriptions from the dropdown
2. Configure start time, time span, and optional criteria
3. Click **Search** to retrieve events
4. Click any row to view full event detail
5. Use **Export CSV** to download results

Custom List Checker

Validate custom list configurations from ZAP files or MEC mappings, and export configurations directly from the datagrid.

Table of contents

Overview

The Custom List Checker validates that CMS100MI/MDBREADMI transactions referenced in custom lists are properly configured. It supports both ZAP file upload and direct MEC mapping selection, and includes a built-in Export Config feature to bulk-export related M3 configurations.

Capabilities

ZAP File Upload

- Upload ZAP configuration files
- Automatically extracts CMS100MI/MDBREADMI transaction references
- Displays results in a filterable/sortable datagrid

MEC Mapping Lookup

- Fetched from Grid API (`/grid/appui/EC_MT`)
- Select a mapping to auto-extract transaction references
- Loading state indicator while fetching mappings
- Alternative to uploading ZAP files — same datagrid output

Export Config (Bulk Export)

- **Export Config** icon button in the datagrid toolbar (next to Refresh)
- Opens the Bulk Export modal pre-populated with distinct records from the current datagrid
- Automatically splits data into 3 tabs:
  - **CMS015** — Distinct Transaction IDs (TRID)
  - **CMS010** — Distinct Info Browser Categories (IBCA)
  - **CRS021** — Distinct FILE + SOPT combinations (user-defined only, excludes standard numeric options)
- Multi-select rows and export/download in one click
- Downloads configuration zip files from `M3/foundation-rest/file-management/v1/`

Usage

Option A — ZAP File:

1. Click **Upload** and select a ZAP file
2. Review extracted transactions in the datagrid

Option B — MEC Mapping:

1. Select a mapping from the lookup field
2. Transactions are automatically extracted and displayed

Exporting Configurations:

1. Load data via ZAP upload or MEC mapping
2. Click the **Export Config** icon ( ) in the datagrid toolbar
3. The Bulk Export modal opens with pre-populated tabs
4. Switch to the desired tab (CMS015, CMS010, or CRS021)
5. Select rows using checkboxes (or "Select All")
6. Click **Export & Download** to trigger exports and download the zip files
7. Review the result dialog — any failed items are listed by name

The Export Config feature requires the app to be deployed inside H5 for automation to work. File downloads require ION API access.

MEC Mapping Viewer

Browse MEC mappings and view compiled Java source code with syntax highlighting.

Table of contents

Overview

The MEC Mapping Viewer provides a read-only interface to browse all MEC (M3 Enterprise Collaborator) mappings in your environment and inspect their compiled Java source code.

Capabilities

Mapping Browser

- Filterable and sortable datagrid of all MEC mappings
- Shared Grid API service — mappings loaded once, cached across tabs
- Supports both admin and read-only MEC environments

Code Viewer

- Monaco Editor integration for Java source code display
- Syntax highlighting, minimap, and search
- Read-only view of compiled mapping logic

Usage

1. Browse mappings in the datagrid
2. Select a mapping row to load its Java source
3. Use Monaco's built-in search (`Ctrl+F`) to navigate the code

Custom BOD Generator

Generate MEC mapping templates (.zap) for custom and standard BOD integrations.

Table of contents

Overview

The Custom BOD Generator creates ready-to-import `.zap` archives containing MEC mapping templates. It supports three modes:

- **Custom BOD** — Generate from scratch using CUSBOD templates with your noun, verb, direction, and field definitions
- **Standard BOD** — Clone a pre-configured standard BOD template (e.g. PurchaseOrder) with a custom noun prefix
- **Upload & Rename** — Upload any existing `.zap` export from MEC and rename the noun throughout

Custom BOD Mode

Capabilities

- Generates complete mapping folder structure (`.map` , `.xsd` , `.xml` , `mapping.properties`)
- Supports **Sync**, **Acknowledge**, **Process**, **Show**, and **Load** verbs
- Supports **In**, **Out**, and **Error Out** directions
- Full `.map` file content from original MEC templates (variables, links, implementations, sequences)
- Version log implementation block included in all templates
- Noun XSD and XML instance rebuilt with user-defined custom fields

Verb-Direction Constraints

Verb	Allowed Directions
Sync	In, Out
Process	In, Out
Show	Out only
Load	In only
Acknowledge	In, Out, Error Out

Usage

1. Select **Custom BOD** mode
2. Enter a **Noun** name (e.g. `CustomerOrder`)
3. Select a **Verb** from the dropdown
4. Choose a **Data Flow** direction
5. Add noun **Properties** — either manually using the field input, or click the **GenAI** button to generate them from a natural language description
6. Click **Generate BOD** to download the `.zap` file
7. Import the `.zap` into MEC via the standard mapping import

GenAI-Assisted Noun Properties

Click the GenAI sparkle icon next to "Noun Properties" to open the AI prompt panel. Describe the fields you need in plain English:

> *"A warranty claim with: claim ID, item number, customer name, claim date, status, resolution notes, and amount"*

The LLM generates PascalCase property names (e.g. `ClaimId` , `ItemNumber` , `CustomerName`) and populates the field list. You can then manually edit, reorder, add, or remove fields before generating the BOD.

Standard BOD Mode

Overview

Clone a pre-configured standard BOD mapping template with a custom noun. This is used when customers want to use a standard BOD structure (e.g. PurchaseOrder) but with a customer-specific prefix (e.g. `CustPurchaseOrder`).

Available Templates

Template	Description
PurchaseOrder - Sync Out	Standard M3 PurchaseOrder outbound sync mapping

Usage

1. Select **Standard BOD** mode
2. Choose a **Base Template** from the dropdown
3. Enter your **Custom Noun** (e.g. `CustPurchaseOrder`)
4. Click **Generate Standard BOD**
5. The template is lazy-loaded, noun is replaced throughout all files, and the `.zap` downloads

Adding New Standard Templates

See the steering file `.kiro/steering/bod-template-generation.md` for instructions on adding new standard BOD templates.

Upload & Rename Mode

Overview

Upload any `.zap` file exported from MEC and rename the noun throughout all files. This works with any mapping — no pre-configuration needed.

How It Works

1. Select **Upload & Rename** mode
2. Upload a `.zap` file (exported from MEC)
3. The base noun is auto-detected from the filename
4. Confirm or edit the **Base Noun** (the text to find and replace)
5. Enter your **Custom Noun** (the replacement text)
6. Click **Generate Renamed BOD**

Technical Details

- Auto-detects file encoding (UTF-16BE, UTF-16LE, UTF-8) for each file in the zip
- Performs noun replacement on both file contents and filenames
- Re-encodes all XSD and `.map` files as UTF-16LE with BOM for MEC compatibility
- Preserves UTF-8 encoding for `.properties` and `.xml` files

Output Format

All modes produce a `.zap` archive with:

- XSD and `.map` files encoded in **UTF-16LE with BOM** (required by MEC mapper)
- `.properties` and `.xml` files in **UTF-8**
- Folder naming convention: `M3BOD_{Noun}_{Verb}_{Direction}_{Version}_Custom`

The `.zap` file can be imported directly into MEC via the standard mapping import workflow.

Data Catalog Registration

After generating a `.zap` in any mode, the tool prompts you to register the object schema in the **ION Data Catalog** (`DATAFABRIC/datacatalog/v1`).

Flow

1. After successful `.zap` download, a confirmation dialog asks: "*Would you like to register in the Data Catalog?*"
2. If you click **Yes**, the tool checks whether the noun already exists
3. If it exists, a warning dialog confirms whether to overwrite
4. On confirmation, a minimal noun metadata XML and XSD payload are built and POSTed to the Data Catalog API

What Gets Registered

- **Noun metadata XML** — includes noun name, verb, identifier path, accounting entity path, and location path
- **Noun schema XSD** — UTF-8 encoded XSD with all user-defined fields as `xs:string` elements

Notes

- For **Custom BOD** mode, the field list from the form is used to build the XSD

- For **Standard BOD** and **Upload & Rename** modes, an empty field list is used (the schema is registered as a placeholder that can be edited later in the Object Schema Editor)
- The user performing the registration is taken from the M3 user context

---

Kiro AI Chat

AI-powered chat assistant with a local WSL backend.

Table of contents

Overview

The Kiro Chat tab provides an AI chat interface powered by the Kiro CLI. It requires a local Node.js backend running in WSL that spawns `kiro-cli` and streams responses back via Server-Sent Events (SSE).

Architecture

```
Angular App (localhost:4200)
  └─ proxy /kiro-chat/* ──▶ Node.js backend (localhost:3000)
                                └─ spawns kiro-cli in WSL
                                └─ streams SSE responses
```

Setup

1. Install WSL

```
wsl --install
```

2. Install Node.js 20+ in WSL

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

3. Install and authenticate Kiro CLI

```
npm install -g kiro-cli
kiro-cli login
```

4. Start the backend

Double-click `start.bat` in the project root, or manually:

```
wsl
cd /mnt/c/Users/<your-user>/Coding/kiro-chat-server
npm install
node server.js
```

5. Start the Angular app

```
ng serve
```

Both servers must be running simultaneously for the chat to work.

Proxy Configuration

The proxy is configured in `odin.json`:

```
{
  "/kiro-chat": {
    "target": "http://localhost:3000",
```

```
    "secure": false,
    "changeOrigin": true,
    "pathRewrite": { "^/kiro-chat": "" }
  }
}
```

Dataflow List

Browse ION dataflows, extract dependencies, and generate DES-030 documents.

Table of contents

Overview

The Dataflow List tab connects to the ION Connect API to list all dataflows in your environment. Selecting a dataflow extracts its full dependency tree — mappings, connection points, file templates, object schemas, scripts, workflows, and custom lists — and can generate a DES-030 design document.

Capabilities

Dependency Extraction

- ION Mappings
- Connection Points (with file template resolution)
- Application Connections
- Splitters & CBR Filters
- Scripts & Workflows
- Object Schemas (nouns)

Custom List Resolution

- Automatically matches APPLICATION nouns to MEC mappings
- Extracts CMS100MI/MDBREADMI transactions from matched mappings
- Fetches custom list data via `EXT100MI.LstCustListData`

DES-030 Generation

- Generates a Word document (`.docx`) with all extracted dependencies
- Configurable user story, title, and change references via a draggable dialog
- Includes connection point details and custom list data

Clipboard Copy

- Copy all grouped dependency info to clipboard in text format

Usage

1. Select a dataflow from the dropdown
2. Review the extracted dependency groups
3. Optionally fill in DES-030 fields (user story, title, change refs)
4. Click **Generate DES-030** to download the Word document, or **Copy** to clipboard

---

GenAI Chat

AI chat interface powered by Infor's GenAI LLM Service with image and document analysis.

Table of contents

Overview

The GenAI Chat tab provides a conversational AI interface that connects to Infor's GenAI backend. It supports session management, message history, streaming responses, and multimodal input (images + documents).

Capabilities

Session Management

- Create new chat sessions
- Browse and resume previous sessions (sorted by last updated)
- View full message history for any session

Streaming Responses

- Real-time streaming of AI responses via SSE (GenAI Chat Service)
- Human and AI messages displayed in a chat bubble UI

Image & Document Upload

- Attach images (PNG, JPEG, GIF, WebP) for AI vision interpretation
- Attach documents (PDF, HTML, TXT, CSV, DOC, DOCX, XLS, XLSX, MD, XML, JSON) for analysis
- Unsupported image formats (e.g. AVIF, BMP) auto-converted to PNG via canvas
- Multiple files per message supported
- Files displayed as chips with filename, size, and remove button

LLM Service Integration

- Uses `GENAI/llmsvc/api/v1/messages` endpoint
- Requires `x-infor-logicalidprefix: lid://infor.genai.genai` header
- Images sent as `{ type: "image", data: "..." }`
- Documents sent as `{ type: "document", data: { format, name, data } }`
- Model: Claude Sonnet 4.5 via Bedrock Converse API

ChatGPT-style Composer

- Sticky bottom rounded box with subtle shadow
- Auto-growing textarea (up to 200px max height, then scrolls)
- Circular ⊕ attach button (left) and send arrow (right)
- Enter sends, Shift+Enter for newline
- Send disabled when no text and no files attached

Usage

1. Type a message and press **Enter** to start a new session
2. Click the ⊕ button to attach images or documents
3. Attached files appear as chips above the textarea — click ✕ to remove
4. Press **Enter** or click the send button to submit
5. Click **Sessions** to browse or resume previous conversations
6. Click **New Chat** to start fresh

Generators

Quick utility generators for common development values.

Overview

The Generators tab provides simple value generators that are frequently needed during M3 development and testing.

Available Generators

Generator	Output	Format
Timestamp	Current date/time	`YYYYMMDDHHmmssCC` (centiseconds)
UUID	Random UUID v4	`xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx`
H5 Script Link	Debug URL for local H5 scripts	`{M3 Base URL}/?scriptCache=false&localScript={url}`

Usage

1. Select a generator from the dropdown
2. Click **Generate**
3. The result is automatically copied to your clipboard

H5 Script Link

Builds a debug URL that loads a local script into the M3 H5 client, bypassing the script cache.

- The M3 base URL is automatically resolved from the user context (`Url` property)
- The **Local Script URL** input is pre-filled with `http://localhost:8080/Script.js` but can be edited
- Useful for testing H5 SDK scripts during development without redeploying

This generator requires the app to be running inside H5 (deployed) so the user context provides the M3 URL.

MEC Variable Validations

Generate null-safe trim statements from MEC Java class fields.

Overview

The MEC Variable Validations tab takes a Java class (typically from a MEC mapping) and generates null-safe trim statements for all `String` fields. This is a common pattern needed in MEC mappings to sanitize input variables.

How It Works

Paste a Java class containing `private String` field declarations. The tool extracts all field names and generates:

```
fieldName = (fieldName == null) ? "" : fieldName.trim();
```

for each field.

Usage

1. Paste your MEC Java class into the input editor
2. Click **Generate**
3. Copy the generated trim statements from the output editor into your mapping

---

Object Schema Editor

Browse, create, edit, and update ION object schema (noun) XSD definitions with GenAI-assisted generation.

Table of contents

Overview

The Object Schema Editor connects to the ION Data Catalog API to list available nouns, load their XSD schema content, register new schemas, and push updates back to the catalog. It includes GenAI integration for natural language schema generation and modification.

Modes

The editor has two modes, selectable via radio buttons:

Add Mode

- Enter a new noun name
- Write XSD manually in the Monaco editor, or use GenAI to generate it from a description

- Click **Register Schema** to add the new noun to the ION Data Catalog
- Includes existence check — if the noun already exists, prompts for overwrite confirmation

Update Mode

- Select an existing noun from the dropdown
- Edit the loaded XSD in the Monaco editor, or use GenAI to describe modifications
- Click **Update Schema** to push changes to the Data Catalog

Capabilities

Noun Browsing

- Lists all nouns from the ION Data Catalog
- Filters by tenant (e.g. shows only tenant-specific nouns for Goldwin environments)
- Refresh button to reload the noun list

Schema Editing

- Monaco Editor with XML syntax highlighting
- Auto-resizing editor (up to 700px height)
- Built-in XML formatter for readability

Schema Registration (Add)

- Register new nouns in the ION Data Catalog
- Auto-generates noun metadata XML with default supported verbs (Show, Get, Update, Acknowledge, Sync, Process, Load)
- Extracts field names from XSD for metadata ID XPath
- Checks for existing nouns before registering

Schema Update

- Push modified XSD back to the ION Data Catalog
- Validation feedback on success or failure
- User context auto-populated for audit trail

Metadata Editing (Update)

In addition to the XSD schema, update mode can edit the noun **metadata** (NounMetadata XML). When a noun is loaded, its metadata is fetched and can be edited via the collapsible **Edit Metadata** panel:

- **Noun Name** — read-only. It identifies the schema being updated and is shown for reference only.
- **Document ID** — a field picker dropdown paired with an editable **ID XPath** text box:
  - The **dropdown** lists every leaf field in the current XSD, traversing nested structures so fields buried inside a repeating or nested element appear as noun-relative paths (e.g. `Data/CONO`, `Data/UUID`); attributes appear as `Element@attr`. Selecting one fills in the ID XPath for you. A **refresh** button reloads the options after you edit the schema.

    - The **ID XPath** text box is the source of truth. Edit it directly for advanced identifiers the picker cannot express, such as predicates and selectors — e.g. `/*/DataArea/Noun/DocumentID[1]/ID[not(@schemeName)]`.
- **Supported Verbs** — checkboxes to toggle Show, Get, Update, Acknowledge, Sync, Process, and Load

The edited metadata is sent together with the XSD when you click **Update Schema**. Previously the fetched metadata was passed through unchanged and could not be edited in the UI.

Document ID validation

Because the XSD can be edited freely, the **entire** ID XPath is validated against the schema structure before an update is sent — not just its last segment. This prevents saving a path whose field name happens to exist but whose overall path is wrong (e.g. `/*/Bogus/DocumentID`).

Update Schema is blocked with an inline error when the path:

- is empty,
- is not an absolute path (must start with `/`),
- has empty segments (e.g. `//`),
- is missing the `DataArea` element,
- does not reference the correct noun element after `DataArea`, or
- contains a plain element/attribute step that is not a declared child of the preceding element in the schema.

When a segment doesn't resolve, the error names the invalid segment and lists the valid child elements at that level, so you can correct the path.

Advanced XPath steps — functions, wildcards (`*`), axis specifiers (`::`), or non-positional predicates like `[not(@schemeName)]` — cannot be checked against the XSD element tree, so they are accepted and deep validation stops at that step. A simple positional predicate such as `DocumentID[1]` is still checked by its element name. If the XSD cannot be parsed into a structure, the deep structural check is skipped entirely (the basic syntax checks still apply) so a legitimate save is not blocked.

GenAI Assist

- Toggle the GenAI panel via the sparkle icon button
- Describe your schema or modification in natural language
- In **Add mode**: generates a complete XSD from your description (uses noun name as root element)
- In **Update mode**: modifies the currently loaded XSD based on your instructions
- Powered by Infor GenAI LLM Service (Claude via ION API)

Usage

Adding a New Schema

1. Select **Add** radio button
2. Enter a noun name (e.g. `CustomWarrantyClaim`)
3. Click the **GenAI** icon and describe your schema (e.g. "Create a schema with fields: claimId, itemNumber, customerName, claimDate, status, description")
4. Click **Generate** — the XSD populates in the editor
5. Optionally click **Format XML** to pretty-print
6. Click **Register Schema** to add to the Data Catalog

Updating an Existing Schema

1. Select **Update** radio button
2. Select a noun from the dropdown
3. Optionally click the **GenAI** icon and describe your change (e.g. "Add a Status field with enum values Active and Closed")
4. Click **Generate** — the XSD is modified in place
5. Optionally click **Edit Metadata** to change the Noun Name, ID XPath (document identifier), or Supported Verbs
6. Click **Update Schema** to push changes (both the XSD and any metadata edits are sent)

Replacing the Document ID

1. Select **Update** and load the noun
2. Click **Edit Metadata**
3. If you have just edited the schema, click the **refresh** button next to the Document ID dropdown to reload the field list
4. Pick the new identifier from the **Document ID** dropdown (the list is drawn from the fields in the current XSD); the full XPath is set for you
5. Click **Update Schema** — the derived ID XPath is validated against the schema before the update is sent

---

Log Configuration

View and manage EC/MEC logger levels directly from the app.

Table of contents

Overview

The Log Configuration tab connects to the Grid API's `LogConfigurationPage` to fetch all registered logger classes and their current log levels. Change any logger's level directly via the inline dropdown editor in the datagrid.

Features

- **Inline dropdown editor** — Click the Level column on any row to select DEBUG, DIAG, INFO, WARN, ERROR, or FATAL
- **Loading indicator** — Busy indicator shown during API calls
- **Filter/search** — Keyword search and filter row to find specific logger classes
- **Soho datagrid** — Sortable, filterable, paginated grid with row height options

How it works

1. On tab load, the app POSTs to `/grid/appui/EC_MT` with the `LogConfigurationPage` navigation target
2. The response contains a table of logger classes with their current level and available level options
3. When you change a level via the dropdown editor, it POSTs the same endpoint with `do: "class"`, `class: "<name>"`, and `level: "<level>"` params

Use cases

- **Troubleshooting MEC mappings** — Set a specific mapping's generated class to DEBUG to capture detailed execution logs
- **Enabling event tracing** — Turn on DEBUG for `com.infor.ec.grid.rest.ProcessREST` to trace document processing
- **Production cleanup** — Quickly reset loggers back to INFO/WARN after debugging

Available log levels

Level	Description
DEBUG	Verbose output for development/troubleshooting
DIAG	Diagnostic-level output
INFO	Standard operational messages
WARN	Potential issues that don't block execution
ERROR	Failures that need attention
FATAL	Critical failures

Export Configurations

Automate M3 configuration exports and downloads via H5 Form Automation.

Table of contents

Overview

The Export Configurations tab provides a streamlined workflow for exporting M3 configuration data in bulk. Click **"Open Export Manager"** to open the modal with tabbed datagrids where you can select and export/download multiple configurations at once.

The modal uses H5 SDK `FormService.executeCommand('RUN', automationXml)` to trigger configuration exports, and the M3 Foundation REST file-management API to download the resulting zip files.

Export Manager Modal

The modal has 5 tabs:

CMS015 — Custom Lists

- Data from `CMS015MI/LstCustomMI`
- Multi-select rows, then Export & Download

CMS010 — Info Browser

- Data from `CMS010MI/LstInfoCat`
- Multi-select rows, then Export & Download

CRS021 — Sorting Options

- Enter a TABLE filter (e.g., `MITMAS`) and click "Load"
- If left empty, loads all tables but shows **user-defined only** (SOPT starting with a letter, not standard numeric ones)
- Multi-select rows, then Export & Download

MDBREADMI — DB Read MI

- Data from `MRS001MI/LstTransactions` filtered by `MINM=MDBREADMI`
- Lists all transactions defined under the MDBREADMI program with transaction name, description, version, status, type, and dates
- Auto-loads on modal open (no filter required)

- Export filename format: `MDBREADMI_MRS010<TransactionName>_<YYYYMMDD>`
- Automation navigates MRS010, positions to MDBREADMI program, selects the transaction, and runs export option 26
- Multi-select rows, then Export & Download

CMS047 — Alert Rules

- Data from `CMS045MI/LstAlertRules`
- Lists alert rules with Publisher (EVPB), Event Name (EVNM), Operation (EVNO), Alert Rule ID (ARID), Status, Description, Name
- Auto-loads on modal open
- Export uses CMS047 program with 4 keys: W1OBKV=EVPB, W2OBKV=EVNM, W3OBKV=EVNO, W4OBKV=ARID
- Only 2 fields: FIELD1 = `SAAGPR_CMS047_<EVPB>_<EVNM>_<EVNO>_<ARID>`, FIELD2 = date
- Multi-select rows, then Export & Download

Actions Per Tab

Button	Behavior
Export & Download	Exports all selected, waits, checks file listing, downloads available files
Export Only	Runs the automation for each selected row without downloading
Download Only	Downloads zip files for selected rows (must already be exported)

Pre-check Before Download

Before downloading, the modal calls `GET M3/foundation-rest/file-management/v1/files%2Fconfig_data%2F` to list available files. Only files that exist are downloaded — missing ones are reported in the result dialog.

Result Dialog

On completion, shows a summary with any failed/missing items listed by name:

- `exported` → automation ran but file not ready yet
- `not ready` → file not found in listing (retry later)
- `export error` → automation failed
- `download error` → file download failed

Custom List Checker Integration

The Custom List Checker tab has an **Export Config** icon button in the datagrid toolbar. Clicking it opens the same Export Manager modal, pre-populated with distinct records from the current datagrid dataset:

- Records with `IBCA = 'MDBREADMI'` populate the **MDBREADMI** tab (TRID is used as the transaction name)
- All other records with non-empty `IBCA` populate the **CMS010** tab
- Records with non-MDBREADMI `TRID` values populate the **CMS015** tab
- Records with user-defined `SOPT` (starting with a letter) populate the **CRS021** tab

Environment Configuration

For local development, set your ION API Bearer token in `src/environments/environment.ts`:

```
export const environment = {
  production: false,
  ionApiDevToken: "eyJraWQ...", // Paste your token here
};
```

This file is gitignored. The token is set once at app startup via `APP_INITIALIZER` — no per-component configuration needed. When deployed to H5, the token comes from the Grid session automatically.

Technical Details

- Automation execution: `FormService.executeCommand('RUN', automationXml, { BMREQ: '' })`
- File download: `IonApiService.execute({ url: 'M3/foundation-rest/file-management/v1/...', responseType: 'blob' })`
- File listing: `GET M3/foundation-rest/file-management/v1/files%2Fconfig_data%2F`
- All modals are draggable by header with viewport containment

This feature requires the app to be running inside H5 (deployed) for the Form Automation to work. File downloads also require ION API access (either via H5 Grid session or a development token for localhost).

Grid API Session Cache

Cache Grid API results in `sessionStorage` for instant loading with tenant-scoped keys.

Table of contents

Overview

The Grid API can take a long time to respond, especially when fetching MEC mappings. To avoid re-fetching every time the app loads or when navigating between tabs, `GridApiService` now caches results in `sessionStorage` with tenant-scoped keys.

How it works

1. On first call, `getMappings()` fetches data from the Grid API and stores it in `sessionStorage`
2. On subsequent calls (same browser tab), data is returned instantly from cache
3. The cache key includes the tenant ID to prevent cross-tenant data leaks in multi-tenant environments
4. When the user closes the browser tab or logs out, `sessionStorage` is automatically cleared by the browser

Cache key format

```
grid_api_cache_{tenant}_mappings
```

Example: `grid_api_cache_MYCOMPANY_DEV_mappings`

The tenant is resolved from the user context (`ctx.tenant` or `ctx.currentCompany`).

Refreshing data

To fetch fresh data from the Grid API (bypassing cache), click the **Refresh** button in the app header (More menu → Refresh). This clears the session cache and reloads the page, forcing a fresh fetch from the Grid API.

Programmatically:

- Call `getMappings(true)` — the `bypassCache` parameter forces a fresh API call
- Call `clearCache()` — clears both in-memory and session cache for mappings
- Call `clearSessionCache()` — removes all grid API cache entries from sessionStorage

Cache lifecycle

Event	Behavior
First app load	Fetches from Grid API, stores in sessionStorage
Subsequent tab visits	Returns from sessionStorage instantly
`getMappings(true)` called	Bypasses cache, fetches fresh, updates cache
`clearCache()` called	Removes mapping entry from memory + session
Browser tab closed	sessionStorage cleared automatically
User logs out of M3	Session ends, cache cleared
Different tenant	Separate cache key, no cross-contamination

Storage limits

If `sessionStorage` quota is exceeded, the service clears all grid API cache entries and retries. This is handled gracefully without errors surfacing to the user.

MEC Map Download

Download MEC mappings as .zap files via the IEC MapGen REST API.

Table of contents

Overview

The MEC Map Download tab allows you to download any MEC (M3 Enterprise Collaborator) mapping directly from the IEC MapGen server as a `.zap` archive file. The downloaded `.zap` is fully compatible with the Business Document Mapper's import function, making it easy to back up, transfer, or version-control mappings.

How It Works

1. **Select a mapping** from the lookup (reuses the cached mapping list from Grid API)
2. **Click Download** to trigger the full download pipeline:
    - Fetches the mapping UUID from the MapGen REST API
    - Downloads the mapping via `GET /M3/iec/ecmapgen/v1/mapping/{name}/{version}/get/{uuid}`
    - Decodes the `ZippedMappingUnit` response (base64 → gzip decompress → UTF-16 decode)
    - Extracts the .map file, schema .xsd files, and mapping.properties
    - Packages everything into a `.zap` archive (ZIP format with DEFLATE compression)
3. **Browser downloads** the resulting `.zap` file

Output .zap Structure

The generated `.zap` file matches the format produced by the Mapper's export function:

```
{MappingName}.zap
├── {MappingName}.map          — Mapping definition (UTF-16 XML)
├── {Schema1}.xsd              — Input/output schema files
├── {Schema2}.xsd
└── mapping.properties         — Mapping metadata
```

Prerequisites

- Your user must have the **M3EC-ClientDesigntime** security role
- The ION API must be connected (`odin login -c <ionapi-file>`)
- The IEC MapGen context root must be accessible at `M3/iec/ecmapgen`

Technical Details

API Endpoints Used

Step	Endpoint	Purpose
1	`GET .../mappings`	List all mappings to find UUID
2	`GET .../{name}/{version}/get/{uuid}`	Download mapping as ZippedMappingUnit

Decoding Pipeline

```
API Response XML
  └─ Extract <cdata> content (base64 string)
       └─ Base64 decode → compressed bytes
            └─ GZip decompress (DecompressionStream API)
                 └─ UTF-16LE decode (skip BOM)
                      └─ MappingUnit XML
```

.zap File Generation

Uses JSZip to create the archive with DEFLATE compression. The `.map` file is encoded as UTF-16LE with BOM to match the Mapper's native format.

Download Log

The component displays a real-time log showing each step of the process, including file names and sizes for each extracted artifact. This helps verify the download completed successfully and shows the mapping's composition.

Architecture

Technical overview of the application structure and services.

Project Structure

Table of contents

Directory Layout

```
src/app/
├── components/
```

```
│   ├── main/                          # Root layout component
│   └── tabs/                          # Feature tabs
│       ├── custom-bod-generator/
│       ├── custom-list-checker/
│       ├── dataflow-list/
│       ├── event-hub-formatter/
│       ├── export-configurations/
│       ├── genai-chat/
│       ├── generators/
│       ├── kiro-chat/
│       ├── log-configuration/
│       ├── mec-map-download/
│       ├── mec-mapping-viewer/
│       ├── mec-variable-validations/
│       ├── object-schema-editor/
│       └── xtendm3-crud-generator/
├── generators/
│   └── des030/                        # DES030 code generators
├── service/                           # Shared services
└── app.module.ts                      # Root module

docs-site/
├── docs/
│   ├── architecture/                  # Architecture documentation
│   ├── configuration/                 # Configuration guides
│   ├── features/                      # Feature documentation pages
│   ├── getting-started/               # Getting started guides
│   └── changelog.md                   # Published changelog
├── generate-pdf.js                    # PDF generator script
├── index.md                           # Home page
└── package.json                       # Docs-site dependencies (md-to-
pdf)
```

Key Dependencies

Package	Purpose
`@infor-up/m3-odin`	M3 API client and utilities
`@infor-up/m3-odin-angular`	Angular bindings for M3 Odin
`ids-enterprise-ng`	Infor Design System (Soho) components
`monaco-editor`	Code editor for MEC viewer and generators
`jszip`	ZIP file generation for downloads
`fast-xml-parser`	XML parsing for ZAP/BOD files
`file-saver`	Client-side file downloads
`docx`	Word document generation

Documentation Site

The `docs-site/` directory is a Jekyll-based documentation site published separately. It also includes a PDF generation script.

PDF Generation

`docs-site/generate-pdf.js` combines all documentation markdown files into a single PDF using `md-to-pdf`. It:

1. Reads files in navigation order (matching Jekyll `nav_order`)
2. Strips Jekyll front matter, Kramdown classes, and Liquid tags
3. Converts internal links (Liquid `link` tags and relative paths) to plain text so they don't resolve to localhost in the PDF
4. Preserves external `https://` links as clickable
5. Outputs `reusable-assets-docs.pdf`

Run with:

```
cd docs-site
npm run pdf
```

Services

Shared Angular services that provide M3 connectivity and business logic.

Table of contents

MI API Service

`mi-api.service.ts`

Wraps `@infor-up/m3-odin` to call M3 API (MI) transactions. Used by most tabs for CRUD operations against standard and custom APIs.

Grid API Service

`grid-api.service.ts`

Connects to the M3 Grid API for fetching MEC mappings and other grid-based data. Results are cached across tabs to avoid redundant network calls.

ION Connect Service

`ion-connect.service.ts`

Handles ION API Gateway connectivity including OAuth token management and endpoint routing.

ION Data Catalog Service

`ion-data-catalog.service.ts`

Interfaces with the ION Data Catalog for metadata and schema operations.

User Context Service

`user-context.service.ts`

Provides the current M3 user context (company, division, user ID). Auto-populated from the M3 session when running inside H5.

Message Service

`message.service.ts`

Centralized notification/messaging service for displaying alerts and status messages across components.

Kiro Chat Service

`kiro-chat.service.ts`

Manages SSE connections to the local Kiro backend, handling message streaming and connection lifecycle.

GenAI Chat Service

`genai-chat.service.ts`

Service layer for the GenAI chat integration.

DES030 Generator Service

`des030-generator.service.ts`

Business logic for generating DES-030 design documents.

Configuration

Environment and proxy configuration for development and deployment.

---

odin.json Configuration

Table of contents

Overview

The `odin.json` file configures the H5 SDK project name and proxy rules for local development. Proxy entries route API calls to the M3 Cloud environment during `ng serve`.

Structure

```
{
  "projectName": "reusable-assets",
  "proxy": {
    "/m3api-rest": {
      "target": "https://mingle-ionapi.
<region>.inforcloudsuite.com/<TENANT>",
      "secure": false,
      "changeOrigin": true
    },
    "/mne": {
      "target": "https://<mne-host>:8080",
      "secure": false,
      "changeOrigin": true
    },
    "/ca": {
      "target": "https://<ca-host>:8080",
      "secure": false,
      "changeOrigin": true
    },
    "/kiro-chat": {
      "target": "http://localhost:3000",
      "secure": false,
      "changeOrigin": true,
      "pathRewrite": { "^/kiro-chat": "" }
    }
  }
}
```

Proxy Entries

Path	Target	Purpose
`/m3api-rest`	ION API Gateway	M3 API (MI) calls
`/mne`	MNE server	MEC/Event Hub endpoints
`/ca`	CA server	Grid API / Data Catalog
`/kiro-chat`	localhost:3000	Local Kiro CLI backend

Deployment

When deployed as an H5 widget inside M3, proxy rules are not needed — the application runs within the same origin as M3 and inherits the session context.

Kiro Chat Setup

Table of contents

Prerequisites

- Windows with WSL installed
- Node.js 20+ inside WSL
- Kiro CLI installed and authenticated

Step-by-step

1. Install WSL

```
wsl --install
```

Restart if prompted.

2. Install Node.js in WSL

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

3. Install Kiro CLI

```
npm install -g kiro-cli
kiro-cli login
```

Follow the browser-based OAuth flow.

4. Start the backend

Use `start.bat` in the project root, or:

```
wsl
cd /mnt/c/Users/<your-user>/Coding/kiro-chat-server
npm install
node server.js
```

The backend starts on `http://localhost:3000`.

5. Verify

With both `ng serve` and the backend running, navigate to the **Kiro Chat** tab at http://localhost:4200.

Both servers must be running simultaneously for the chat to function.

Changelog

All notable changes to this project are documented below.

2026-09-30

Added: Object Schema Editor — Metadata Editing

- Update mode now loads and edits the noun metadata XML (NounMetadata) alongside the XSD schema
- New collapsible **Edit Metadata** panel to edit Noun Name, ID XPath (document identifier), and Supported Verbs
- Replace the last segment of the ID XPath to change which field is used as the document ID
- Edited metadata is sent in the update payload (previously the fetched metadata was passed through unchanged)
- Added `getNounMetadataXml()` and `buildNounPayloadWithMetadata()` to the data catalog service

Changed: Object Schema Editor — Read-only Noun Name & Document ID Validation

- Noun Name is now read-only in the metadata panel
- The Document ID combines a field picker dropdown with an editable ID XPath text box; the text box is the source of truth and supports advanced XPath (predicates/selectors like `DocumentID[1]/ID[not(@schemeName)]`) the picker cannot express
- The dropdown traverses nested structures, listing fields as noun-relative paths (e.g. `Data/CONO`) so deeply nested fields are selectable, not just the noun's direct children
- Fixed a crash that prevented the Edit Metadata panel from opening — the field picker is now a native select instead of a Soho dropdown, which also makes the refresh button reflect schema changes immediately
- The metadata panel now opens whenever a noun is selected, even if its metadata XML failed to load
- The whole ID XPath is validated against the schema structure on save (not just the last segment); advanced XPath steps (functions, wildcards, non-positional predicates) are accepted without structural checking so valid selector-based IDs are not blocked
- Inline error messages name the invalid segment and list the valid children at that level

---

2026-07-17

Improved: MEC Map Download — Session Storage Caching

- Mapping list is now cached in session storage (same pattern as Custom List Checker's Grid API caching)
- Extracted caching logic into `MecMapCacheService` for reuse and preloading
- Mappings are preloaded at application startup so the list is ready when the tab is opened
- Header toolbar "Refresh" button clears MEC mapping cache alongside Grid API cache

- Deduplicates concurrent requests via a shared loading promise

---

2026-07-10

Added: MDBREADMI Export Tab

- New "MDBREADMI — DB Read MI" tab in the Bulk Export Modal
- Lists all MDBREADMI transactions via `MRS001MI/LstTransactions`
- Supports Export & Download, Export Only, and Download Only with multi-select
- Automation navigates MRS010, selects MDBREADMI program and transaction, exports via option 26
- Export filename: `MDBREADMI_MRS010<TransactionName>_<YYYYMMDD>`
- Auto-loads transaction list on modal open

Changed: Export Configurations Simplified

- Removed single-export dropdown and free-text fields — all exports now go through the bulk modal
- Reduces errors since values come from actual M3 data rather than manual entry

Changed: Custom List Checker — MDBREADMI Separation

- Records with `IBCA = 'MDBREADMI'` now populate the MDBREADMI tab instead of CMS010
- CMS015 tab excludes MDBREADMI transactions for cleaner separation

Added: CMS047 Export Tab (Alert Rules)

- New "CMS047 — Alert Rules" tab in the Export Manager modal
- Lists alert rules via `CMS045MI/LstAlertRules` (Publisher, Event Name, Operation, Alert Rule ID)
- Automation navigates CMS047 with 4 keys (EVPB, EVNM, EVNO, ARID), exports via option 26
- FIELD1: `SAAGPR_CMS047_<EVPB>_<EVNM>_<EVNO>_<ARID>`

2026-07-09

New: MEC Map Download Tab

- Download MEC mappings as .zap files via the IEC MapGen REST API
- Select mapping from lookup, download via ION API Gateway, decode and extract to .zap archive
- Output includes .map file (UTF-16), schema .xsd files, and mapping.properties
- Compatible with MEC Mapper's import function
- Real-time download log shows extraction progress

Enhanced: Object Schema Editor — GenAI Assist & Add Mode

- Added radio button toggle to switch between "Add" and "Update" modes
- **Add mode**: Enter a new noun name, use GenAI to describe and generate XSD, then register the schema in ION Data Catalog
- **Update mode**: Select existing noun from dropdown, use GenAI to describe modifications, then update the schema
- GenAI button (sparkle icon) toggles a collapsible prompt panel for natural language schema generation/modification via Infor GenAI LLM Service
- Register Schema flow includes existence check with overwrite confirmation
- Automatically extracts field names from generated XSD for noun metadata registration

Enhanced: Custom BOD Generator — GenAI-Assisted Noun Properties

- Added GenAI button next to "Noun Properties" label in the Custom BOD form
- Describe noun properties in natural language and the LLM generates PascalCase field names
- Generated fields populate the list and can be manually edited, reordered, or removed

---

2026-07-08

New: Export Configurations Tab

- Single-item export: Select program (CMS015/CMS010/CRS021), fill in parameters, execute automation and download zip
- Uses `FormService.executeCommand('RUN', automationXml)` — the working MUA pattern for H5 Form Automation
- Download via `M3/foundation-rest/file-management/v1/` (ION API)
- Preview of generated FIELD1 and FIELD2 values before execution

New: Bulk Export Modal

- Accessible from Export Configurations tab ("Bulk Export" button) and Custom List Checker datagrid toolbar (export icon)
- 3 tabbed datagrids: CMS015 (Custom Lists), CMS010 (Info Browser), CRS021 (Sorting Options)
- Multi-select with checkbox column and "Select All" header
- CMS015 loads from `CMS015MI/LstCustomMI`
- CMS010 loads from `CMS010MI/LstInfoCat`
- CRS021 has collapsible accordion filter for TABLE name; loads user-defined sorting options only (SOPT starting with letter)
- Three actions per tab: Export & Download, Export Only, Download Only
- Pre-checks file listing API (`GET files/config_data/`) before downloading with retry logic
- Detailed failure dialog listing each missing/failed config by name and status

- Per-tab loading indicators
- Standard Soho toolbar with More button (personalize columns, row height, filter)
- When opened from Custom List Checker, pre-populates with distinct records from current datagrid

Enhanced: All Component Modals (Draggable)

- Bulk Export, Custom List Export, and DES-030 modals are now draggable by header
- Native mousedown/mousemove implementation with viewport containment
- Delayed positioning (500ms) to account for data loading
- max-height: 90vh with internal scrolling

Added: Environment-based Dev Token

- `src/environments/environment.ts` with `ionApiDevToken` for local ION API development
- Single `APP_INITIALIZER` sets token once for all components
- Removed all per-component `setDevelopmentToken` / `isLocalhost` calls
- File gitignored to prevent token commits

Enhanced: Custom List Checker Tab

- Added "Export Config" icon button in datagrid toolbar
- Opens Bulk Export modal pre-populated with current dataset
- Fixed file upload event type (`SohoFileUploadEvent` → `any`)

Docs

- Added Export Configurations feature page
- Updated Custom List Checker docs with export functionality
- Updated Getting Started with ION API token setup instructions
- Updated Dataflow List docs to mention draggable DES-030 dialog
- Updated index.md feature table descriptions

2026-07-07

Enhanced: Grid API Service — Session Cache with Tenant-Scoped Keys

- `GridApiService.getMappings()` now checks `sessionStorage` before making Grid API calls
- Cache key includes tenant ID for multi-tenant safety (e.g., `grid_api_cache_TENANT_mappings`)
- Data persists for the browser tab session — automatically cleared on tab close or logout
- `getMappings(bypassCache)` accepts an optional `bypassCache` flag to force a fresh fetch

- `clearCache()` now also removes the session entry alongside the in-memory cache
- New `clearSessionCache()` method to wipe all grid API cache entries at once

2026-07-02

Enhanced: App Startup — Pre-load Grid API Data

- Grid API mappings are now fetched in the background at app startup (after user context is loaded)
- MEC Mapping Viewer, Custom List Checker, Dataflow List, and Log Configuration tabs load instantly since data is already cached

Enhanced: Event Hub Formatter Tab

- **Selected subscriptions float to top** — Uses Soho dropdown's built-in `moveSelected: 'all'` to show selected items at the top of the list

Enhanced: XtendM3 CRUD Generator Tab

- **Full-screen code preview modal** — Replaced inline bottom panel with a centered modal overlay (90vw × 85vh) for focused code editing
- **Editable code** — Monaco editors are now read-write; edits are preserved per tab and used when uploading or downloading
- Modified tab indicator (blue dot) shows which transactions have been manually edited
- Upload and Download buttons in the modal header for quick actions directly from the preview
- Clicking outside the modal (on the overlay) closes it
- Download and Upload from toolbar also use edited code when preview tabs have modifications

Enhanced: Navigation — Module Nav Sidebar with Angular Router

- Replaced vertical Soho tabs with a custom module-nav sidebar (IDS-aligned spacing and styling)
- Icons for each nav item with IDS icon set (document, search-list, calendar, script, etc.)
- Collapsible sidebar: expanded (300px with labels) or collapsed (56px icon rail) via hamburger toggle
- Search/filter for nav items — filters as you type, clears on X button
- Angular Router integration with hash routing (`useHash: true`) for H5 SDK compatibility
- `CacheRouteReuseStrategy` preserves component state across navigation (no reset on tab switch)
- Sidebar and header on same row — sidebar spans full viewport height, header only in content area

Enhanced: Theme Support

- **Theme switching** — Mode (Light/Dark/Contrast) and Color (Default, Amber, Amethyst, Azure, Emerald, Graphite, Ruby, Slate, Turquoise) via submenu in More button
- Checkmark indicator on currently active mode and color
- `ThemeService` — shared service managing Monaco editor theme globally

- Monaco editors respond to theme changes across all cached routes (re-applied on navigation)
- All custom CSS uses `inherit`, `currentColor`, and `rgba()` — fully theme-responsive
- Chat components (Kiro Chat, GenAI Chat) styled with semi-transparent colors for dark mode support

Refactored: App Architecture

- `AppComponent` stripped to thin shell (user context + locale init only)
- Header, navigation, theme, and about modal moved to `MainComponent`
- Removed dead code: old personalize handlers, unused ViewChild refs, duplicate about modal
- Cleaned XtendM3 code preview CSS to be theme-responsive
- MEC Variable Sanitizer: removed dynamic editor resize (fixed height with internal scroll)

Enhanced: MEC Variable Sanitizer

- Fixed overlapping Monaco editors — both now use fixed 300px height with internal scrolling
- Instructions box styled with `rgba()` for dark mode compatibility
- Removed content-based auto-resize that caused layout issues

---

2026-06-20

New: Log Configuration Tab

- Browse all EC/MEC logger classes with their current log levels in a Soho datagrid
- Inline dropdown editor on the Level column to change log level per class (DEBUG, DIAG, INFO, WARN, ERROR, FATAL)
- Loading indicator shown during API calls
- Uses Grid API `LogConfigurationPage` endpoint with CSRF token handling
- Full Soho toolbar with keyword search, filter, row height, and personalize columns

---

2026-05-28

Enhanced: GenAI Chat Tab

- **Image & Document upload** — Attach images (PNG, JPEG, GIF, WebP) and documents (PDF, HTML, TXT, CSV, DOC, DOCX, XLS, XLSX, MD, XML, JSON) for AI interpretation
- Unsupported image formats (e.g. AVIF) auto-converted to PNG via canvas
- ChatGPT-style composer UI: sticky bottom box with auto-grow textarea, file chips, circular attach/send buttons
- Multiple files can be attached per message

- Uses Infor GenAI LLM Service (`GENAI/llmsvc/api/v1/messages`) with `x-infor-logicalidprefix` header
- File type auto-detection: images sent as `image` blocks, documents sent as `document` blocks with format/name metadata

Enhanced: Kiro Chat Tab

- Redesigned UI to match GenAI Chat — same ChatGPT-style composer with file chips, auto-grow textarea, and attach/send buttons
- File attachment support added (visual only — files shown as chips)

Chore

- Refactored code structure for improved readability and maintainability
- Added changelog to docs-site
- Linked Key Features table on docs home page to individual feature pages

---

2026-05-26

Enhanced: Custom BOD Generator Tab

- **Auto-register in Data Catalog** — After generating a .zap, prompts user to register the object schema in the Data Catalog via `DATAFABRIC/datacatalog/v1`
- Checks for existing noun before creating; warns and confirms overwrite if already present
- Builds minimal noun metadata XML and UTF-8 XSD payload for all three modes (Custom, Standard, Upload & Rename)
- Migrated Data Catalog API from deprecated `IONSERVICES` to `DATAFABRIC` endpoint
- **Standard BOD mode** — Select from pre-configured standard BOD templates (e.g. PurchaseOrder Sync Out) and rename with a custom noun prefix
- **Upload & Rename mode** — Upload any .zap file exported from MEC, specify the base noun to replace, and generate a renamed copy with your custom noun
- Auto-detects encoding (UTF-16BE, UTF-16LE, UTF-8) when reading uploaded files
- Auto-detects base noun from uploaded .zap filename
- Standard BOD templates are lazy-loaded (only downloaded when selected)
- Template data embedded as JSON to avoid H5 server 403 restrictions on static assets
- Added version log implementation block to all custom BOD .map templates
- Three-mode UI: Custom BOD / Standard BOD / Upload & Rename
- Enhanced upload process by extracting nouns and fields from XSD files

---

2026-05-25

Enhanced: Dataflow List Tab

- Added DES-030 modal for generating export details document

Enhanced: App Shell (About)

- Added attribution for XtendM3 CRUD Generator in about section

---

2026-05-24

New: Generators Tab — H5 Script Link Generator

- Builds a debug URL using the M3 base URL from user context
- Local script URL input pre-filled with `http://localhost:8080/Script.js` (editable)
- Result auto-copied to clipboard

Enhanced: Custom BOD Generator Tab

- Migrated to synchronous template generation with embedded JSON templates
- Removed legacy BOD template assets (previously served as static files)
- Files stored as `.txt` or `.mapxml` to avoid webpack source-map parsing issues and H5 server 403 restrictions

Docs

- Added guidelines for updating documentation when implementing new features or changes

---

2026-05-23

Docs

- Added Custom BOD Generator feature documentation page

2026-05-22

Enhanced: App Shell

- Added documentation link to more actions menu and about section

---

2026-05-21

Enhanced: App Shell

- Added more actions menu with refresh and about buttons

Enhanced: Dataflow List Tab

- DES-030 document generation now supports custom list data and improved data handling

Enhanced: Event Hub Formatter Tab

- Added 'Changes' column to event grid showing element-level change tracking

---

2026-05-17

New: GenAI Chat Tab

- Session management (create, list, select sessions)
- Streaming chat with Infor GenAI service
- Message history display per session

2026-05-15

Enhanced: Dataflow List Tab

- DES-030 DOCX generator with Infor-style formatting (cover page, TOC, connection points, deployment, recovery sections)
- Exclude specific groups and optimize row handling in DES-030 generation
- Added header logo images for document branding

2026-05-13

Enhanced: Dataflow List Tab

- Added loading indicator for dataflow selection

Enhanced: Object Schema Editor Tab

- Added loading indicator for object schema selection

Enhanced: XtendM3 CRUD Generator Tab

- Added loading indicator for configuration selection

Enhanced: Custom List Checker Tab

- Replaced mapping dropdown with lookup input field

Enhanced: MEC Mapping Viewer Tab

- Replaced mapping dropdown with lookup input field

Enhanced: App Shell (About)

- Updated versioning and copyright information
- Added author information

2026-05-12

New: Custom BOD Generator Tab

- Generate CUSBOD mapping templates (.zap) from a noun, verb, direction, and custom field list
- Supports Sync, Acknowledge, Process, Show, and Load verbs with In/Out/Error Out directions
- Dynamic verb-direction constraints (e.g. Show = Out only, Load = In only)
- Full .map file content copied from original MEC templates (includes variables, links, implementations, sequences)
- Version log implementation block included in all templates
- Noun XSD and XML instance rebuilt with user-defined custom fields
- Output files encoded in UTF-16LE with BOM as required by MEC mapper
- Downloads as a ready-to-import .zap archive

New: XtendM3 CRUD Generator Tab

- Generate full CRUD transactions (Add/Del/Get/Lst/Upd) for XtendM3 custom tables
- Upload transactions directly to M3 Extensibility API with auto-activation
- Upload table definitions to Foundation REST `importTables` endpoint (with CSRF)
- Save/load field configurations to M3 environment via EXT999MI
- Download as ZIP or export/import JSON configs
- User field auto-populated from M3 login (readonly when authenticated)
- Enhanced table upload with fallback download option

New: MEC Mapping Viewer Tab

- Browse all MEC mappings in a filterable/sortable datagrid
- View compiled Java source code with Monaco editor (syntax highlighting, minimap, search)
- Shared Grid API service — mappings loaded once, cached across tabs
- Supports both admin and read-only MEC environments

Enhanced: Custom List Checker Tab

- Added MEC mapping dropdown (fetched from Grid API `/grid/appui/EC_MT`)
- Select a mapping to auto-extract CMS100MI/MDBREADMI transactions
- Alternative to uploading ZAP files — same datagrid output
- Loading state indicator on dropdown

Enhanced: Event Hub Formatter Tab

- Browse Event Hub subscriptions directly from the M3 environment
- Multi-select subscriptions with start time, time span, and criteria filters
- Results displayed in a filterable/sortable datagrid
- Row selection opens event detail in a Soho modal dialog
- Export events to CSV with flattened element columns
- Existing CSV/text formatting preserved

2026-05-11

New: Kiro Chat Tab

- Chat component with backend integration (WSL + kiro-cli)
- Session management with create/list/select
- Loading animation for assistant messages
- Added `start.bat` script and README instructions for WSL backend setup

---

2026-04-21

New: Dataflow List Tab

- Browse and inspect ION dataflows
- Copy dataflow info to clipboard with enhanced formatting

2026-04-07

Enhanced: Custom List Checker Tab

- Updated datagrid component and enhanced related tables functionality
- Added JSON configuration for EXT100MI program modules and transactions

---

2026-04-01

Enhanced: Object Schema Editor Tab

- Added clipboard functionality (copy XML to clipboard)
- Added XML formatting/pretty-print

2025-11-25

Chore

- Initialized Angular application template with essential files and configurations

---

2025-10-18

Enhanced: Object Schema Editor Tab

- Fixed `isBusy` implementation — proper RxJS chaining with `switchMap`, `finalize`, and error handling
- Added tenant-based noun filtering (GOLDWIN tenant check)

Chore

- Standardized indentation across TypeScript files

2025-10-13

Enhanced: XtendM3 CRUD Generator (standalone app)

- Added CRUD functionality and personalization features

2025-09-29

Enhanced: XtendM3 CRUD Generator (standalone app)

- Added configuration lookup and save functionality
- Added `programName` and `user` parameters to builder constructors
- Integrated JSZip for file downloads
- Updated API targets to new Infor Cloud Suite endpoint
- Implemented `uploadToAPI` method for uploading generated JSON files

2025-09-28

Initial Release

- Initial commit with project scaffolding
- Added datagrid template for testing
